SOFTWARE ENGINEERING · CHAPTER 04

RESTful Web Services (JAX-RS)

RESTful web services expose resources over HTTP using standardized methods, representations, and status codes. Jakarta REST 4.0 provides the Java API for implementing REST endpoints within Jakarta EE 11; we run it with Eclipse Jersey on an embedded Grizzly server.

Jakarta REST 4.0 **Eclipse Jersey 4 · Grizzly** **Jakarta EE 11** **JDK 25 LTS**

AGENDA

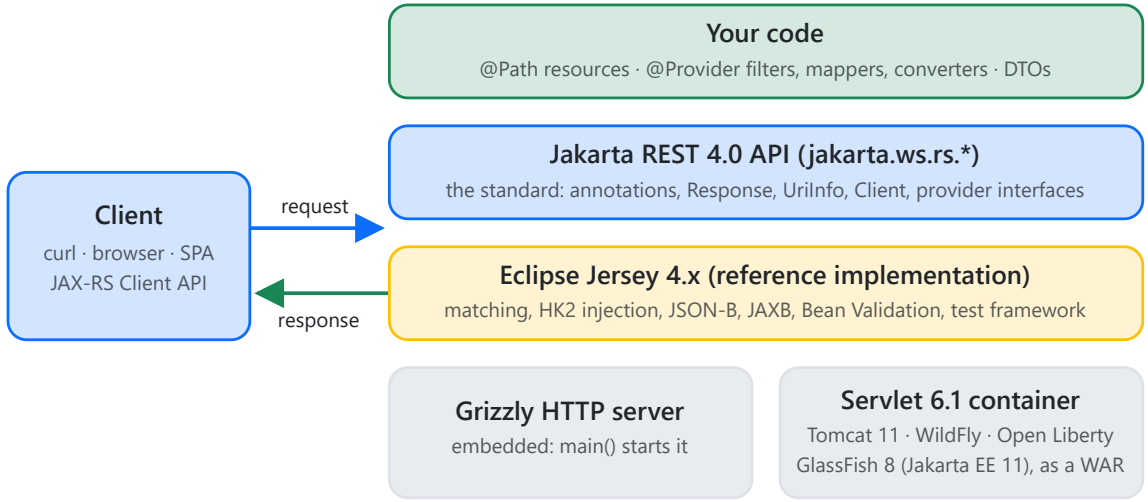
What we will cover

1. Resource-oriented API design
2. HTTP methods, status codes, and headers
3. URI paths, path parameters, and query parameters
4. Content negotiation and JSON or XML representations
5. Request and response entities
6. JAX-RS resources, providers, and filters
7. Validation and consistent error responses
8. Pagination, filtering, and versioning
9. Idempotency, caching, and HATEOAS concepts
10. API documentation, testing, and security
11. Chapter Quiz
12. Lab-04

Click any item to jump directly to that topic.

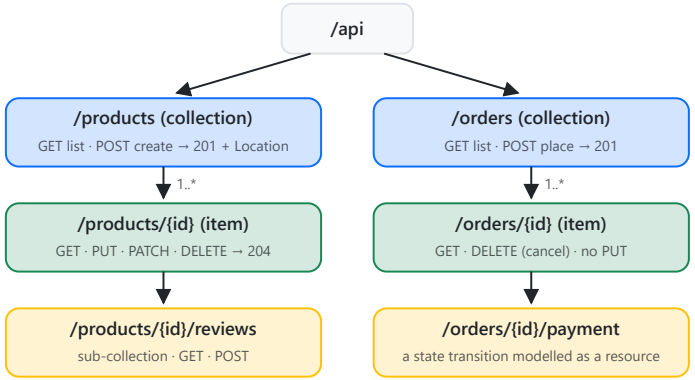
REST, Jakarta REST 4.0 and Jersey: who does what

- **REST** (Fielding, 2000) is an architectural style, not a protocol: *resources* identified by URIs, manipulated through *representations* (JSON, XML) with a *uniform interface* (the HTTP methods), *stateless* requests, *cacheable* responses, layered systems.
- **Jakarta REST 4.0** (formerly JAX-RS, package `jakarta.ws.rs`) is the Java standard in **Jakarta EE 11**: annotations that map HTTP to methods, a `Response` builder, providers, a client API. Jakarta REST 5.0 pairs with the next platform release: study it together with that platform.
- **Eclipse Jersey 4.x** is the reference implementation. In this chapter it runs embedded on **Grizzly**; the same classes deploy unchanged as a WAR to GlassFish 8, WildFly or Open Liberty.
- Chapter 02 built a REST API with Spring MVC. Here you learn the *standard* API and the HTTP rules that both frameworks implement.



Same resource classes on either container: only the bootstrap differs.

Think in resources: nouns in the URI, verbs in the method



Avoid (RPC style)	Prefer (resource style)
`POST /getProduct?id=42`	`GET /products/42`
`POST /products/42/delete`	`DELETE /products/42`
`POST /payOrder`	`PUT /orders/17/payment`
`/Product_List`, `/productList`	`/products` (plural, lower-case, hyphens)

```java
package edu.itc.shop.api;

import java.util.List;
import jakarta.inject.Inject;
import jakarta.ws.rs.*;
import jakarta.ws.rs.core.MediaType;

// A collection resource. The class is instantiated PER REQUEST by default,
// so state lives in an injected singleton (the repository), not in fields.
@Path("/products")
@Produces(MediaType.APPLICATION_JSON)
public class ProductResource {
    private final ProductRepository repo;

    @Inject
    public ProductResource(ProductRepository repo) { this.repo = repo; }

    // GET /api/products
    @GET
    public List<ProductDto> list() {
        return repo.findAll().stream().map(ProductDto::from).toList();
    }

    // GET /api/products/42  (404 when the id is unknown)
    @GET @Path("{id}")
    public ProductDto one(@PathParam("id") long id) {
        return repo.findById(id).map(ProductDto::from)
                .orElseThrow(() -> new NotFoundException(
                        "Product " + id + " not found"));
    }

    // Sub-resource locator: /api/products/42/reviews/...
    @Path("{id}/reviews")
    public ReviewResource reviews(@PathParam("id") long id) {
        return new ReviewResource(repo.require(id));
    }
}
```

Running the same resources: embedded Grizzly, or a WAR on any EE 11 server

```
<!-- pom.xml: Jersey BOM pins every module to one version -->
<properties>
  <maven.compiler.release>25</maven.compiler.release>
  <jersey.version>4.0.0</jersey.version>
</properties>
<dependencyManagement><dependencies><dependency>
  <groupId>org.glassfish.jersey</groupId>
  <artifactId>jersey-bom</artifactId>
  <version>${jersey.version}</version>
  <type>pom</type><scope>import</scope>
</dependency></dependencies></dependencyManagement>
<dependencies>
  <!-- jakarta.ws.rs-api 4.0 arrives transitively -->
  <dependency>
    <groupId>org.glassfish.jersey.containers</groupId>
    <artifactId>jersey-container-grizzly2-http</artifactId>
  </dependency>
  <dependency>
    <groupId>org.glassfish.jersey.inject</groupId>
    <artifactId>jersey-hk2</artifactId>
  </dependency>
  <dependency>
    <groupId>org.glassfish.jersey.media</groupId>
    <artifactId>jersey-media-json-binding</artifactId>
  </dependency>
  <dependency>
    <groupId>org.glassfish.jersey.media</groupId>
    <artifactId>jersey-media-jaxb</artifactId>
  </dependency>
  <dependency>
    <groupId>org.glassfish.jersey.ext</groupId>
    <artifactId>jersey-bean-validation</artifactId>
  </dependency>
</dependencies>
```

```
package edu.itc.shop;

import java.net.URI;
import org.glassfish.grizzly.http.server.HttpServer;
import org.glassfish.jersey.grizzly2.httpserver.GrizzlyHttpServerFactory;
import org.glassfish.jersey.server.ResourceConfig;

public class Main {
    public static void main(String[] args) throws Exception {
        // scans the package for @Path and @Provider classes
        ResourceConfig config = new ResourceConfig()
                .packages("edu.itc.shop.api")
                .register(new ShopBinder());   // repository singleton
        HttpServer server = GrizzlyHttpServerFactory.createHttpServer(
                URI.create("http://localhost:8080/api/"), config);
        System.out.println("Shop API: http://localhost:8080/api/products");
        Thread.currentThread().join();       // Ctrl-C stops the server
    }
}
```

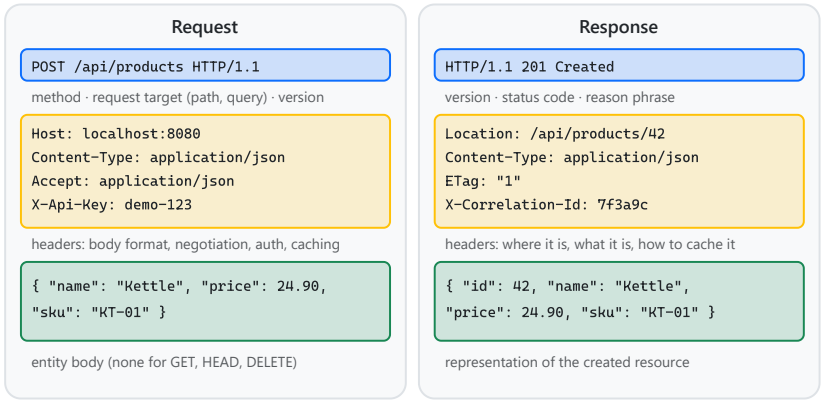
```
// The portable form: package as a WAR, deploy to GlassFish 8,
// WildFly or Open Liberty. The container scans @Path/@Provider itself.
@ApplicationPath("/api")
public class ShopApplication extends Application { }

// Standard Java SE bootstrap (Jakarta REST 3.1+), no Jersey types:
SeBootstrap.start(new ShopApplication(),
        SeBootstrap.Configuration.builder().port(8080).build());
```

💡 `ResourceConfig` extends `Application`. Override `getClasses()` in the WAR variant if you prefer explicit registration over scanning.

TOPIC 2

The anatomy of a request and a response




```java
@Path("/products")
@Produces(MediaType.APPLICATION_JSON)
@Consumes(MediaType.APPLICATION_JSON)
public class ProductResource {

    // POST creates a child of the collection: 201 + Location header
    @POST
    public Response create(ProductDto dto, @Context UriInfo uriInfo) {
        Product saved = repo.save(dto.toEntity());
        URI location = uriInfo.getAbsolutePathBuilder()
                              .path(Long.toString(saved.id())).build();
        return Response.created(location)
                       .entity(ProductDto.from(saved))
                       .build();
    }

    // PUT replaces the whole representation: idempotent, 200 (or 201)
    @PUT @Path("{id}")
    public ProductDto replace(@PathParam("id") long id, ProductDto dto) {
        return ProductDto.from(repo.replace(id, dto.toEntity()));
    }

    // PATCH changes part of it: send only the fields that change
    @PATCH @Path("{id}")
    @Consumes("application/merge-patch+json")
    public ProductDto patch(@PathParam("id") long id, ProductPatch patch) {
        return ProductDto.from(repo.patch(id, patch));
    }

    // DELETE: 204 with no body; a second DELETE gives 404 (still idempotent)
    @DELETE @Path("{id}")
    public Response delete(@PathParam("id") long id) {
        if (!repo.delete(id)) throw new NotFoundException();
        return Response.noContent().build();
    }
}
```

Choosing the status code, reading and writing headers

```
// The Response builder: status + entity + headers, in any order
return Response.status(Response.Status.CONFLICT)
               .type("application/problem+json")
               .entity(new Problem("about:blank", "Conflict", 409,
                       "SKU KT-01 already exists", "/api/products"))
               .build();

// Shortcuts for the common cases
Response.ok(dto).build();                 // 200 with a body
Response.created(location).build();       // 201 + Location
Response.accepted().build();              // 202: work continues later
Response.noContent().build();             // 204: success, no body
Response.seeOther(uri).build();           // 303: go GET this instead
Response.notModified().build();           // 304: your cached copy is fine

// Reading headers: @HeaderParam for one, HttpHeaders for all
@GET @Path("{id}")
public Response one(@PathParam("id") long id,
                    @HeaderParam("X-Request-Id") String requestId,
                    @HeaderParam(HttpHeaders.USER_AGENT) String agent) {
    return Response.ok(find(id))
                   .header("X-Request-Id", requestId)
                   .header(HttpHeaders.CACHE_CONTROL, "max-age=60")
                   .header("Vary", "Accept")
                   .build();
}

@GET @Path("echo")
public Map<String, List<String>> echo(@Context HttpHeaders headers) {
    return headers.getRequestHeaders();
}

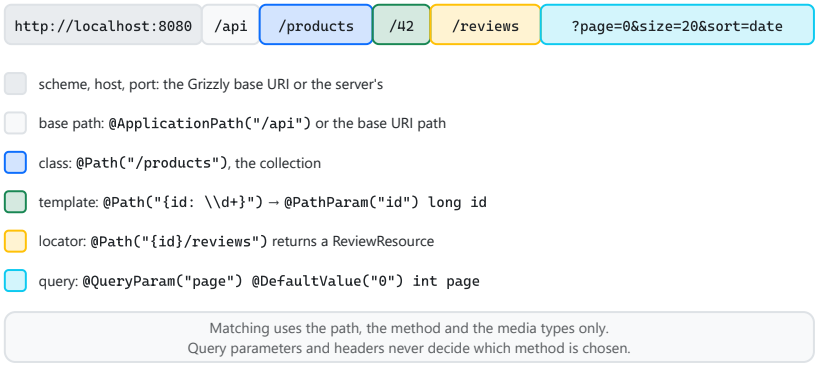
// Errors are exceptions: the runtime turns them into responses
throw new NotFoundException("Product " + id + " not found");    // 404
throw new BadRequestException("size must be 1..100");           // 400
throw new ClientErrorException("Idempotency-Key required", 428);
throw new WebApplicationException(Response.status(429).build());
```

Method	Safe	Idempotent	Body	Typical success
`GET`	yes	yes	no	200, 304
`HEAD`	yes	yes	no	200 (headers only)
`POST`	no	**no**	yes	201 + Location, 202
`PUT`	no	yes	yes	200, 201, 204
`PATCH`	no	no	yes	200
`DELETE`	no	yes	no	204

Code	Meaning: use it when…
400	the request cannot be parsed or a parameter has the wrong type
401 / 403	no or bad credentials / valid credentials, not allowed
404 / 405	no such resource / resource exists, method does not
406 / 415	cannot produce what `Accept` asks / cannot read the sent `Content-Type`
409 / 412	state conflict (duplicate, stock) / precondition (`If-Match`) failed
422 / 428	well-formed but invalid data / a required precondition header is missing
500 / 503	our bug (log it, hide details) / temporarily unavailable, add `Retry-After`

⚠ Never answer `200 OK` with an error message inside the body: caches, monitoring and clients all read the status code first.

From URI to Java parameters: @Path, @PathParam, @QueryParam




```java
@Path("/products")
public class ProductResource {

    // Regex template: /products/42 matches, /products/abc does not (404)
    @GET @Path("{id: \\d+}")
    public ProductDto one(@PathParam("id") long id) { ... }

    // Literal segment before a template; templates may span patterns
    @GET @Path("by-sku/{sku: [A-Z]{2}-\\d{2}}")
    public ProductDto bySku(@PathParam("sku") String sku) { ... }

    // ?category=books&minPrice=5&sort=price,desc&tag=new&tag=sale
    @GET
    public List<ProductDto> search(
            @QueryParam("category") String category,
            @QueryParam("minPrice") @DefaultValue("0") BigDecimal minPrice,
            @QueryParam("sort") @DefaultValue("name,asc") String sort,
            @QueryParam("tag") List<String> tags) {
        return service.search(category, minPrice, sort, tags);
    }

    // Any type with a String constructor, valueOf(String) or a
    // ParamConverter works; a bad value gives 404 (path) or 400 (query)
    @GET @Path("{id}/price")
    public Money price(@PathParam("id") long id,
                       @HeaderParam("Accept-Language")
                       @DefaultValue("en") String lang) { ... }

    // Raw access when you need everything at once
    @GET @Path("debug")
    public String debug(@Context UriInfo info) {
        return info.getRequestUri() + " " + info.getQueryParameters();
    }
}
```

Grouping parameters, delegating paths, converting custom types

```
// @BeanParam: related parameters in one class (fields or setters)
public class ProductFilter {
    @QueryParam("q") String text;
    @QueryParam("category") String category;
    @QueryParam("minPrice") @DefaultValue("0 EUR") Money minPrice;
    @QueryParam("maxPrice") Money maxPrice;
    @QueryParam("page") @DefaultValue("0") @Min(0) int page;
    @QueryParam("size") @DefaultValue("20") @Min(1) @Max(100) int size;
    @HeaderParam("Accept-Language") @DefaultValue("en") String lang;
}

@GET
public List<ProductDto> search(@BeanParam @Valid ProductFilter f) {
    return service.search(f);
}

// ParamConverter: teach the runtime to parse "12.50 EUR" into Money
@Provider
public class MoneyConverterProvider implements ParamConverterProvider {
    @Override
    @SuppressWarnings("unchecked")
    public <T> ParamConverter<T> getConverter(Class<T> raw, Type generic,
                                              Annotation[] annotations) {
        if (raw != Money.class) return null;         // not ours: skip
        return (ParamConverter<T>) new ParamConverter<Money>() {
            public Money fromString(String s) { return Money.parse(s); }
            public String toString(Money m) { return m.toString(); }
        };
    }
}
```

Annotation	Source
`@PathParam`	URI template variable
`@QueryParam`	`?name=value` (repeatable → `List`)
`@HeaderParam` / `@CookieParam`	request header / cookie
`@FormParam`	`application/x-www-form-urlencoded` body
`@MatrixParam`	`/products;color=red`
`@BeanParam`	aggregates any of the above
`@DefaultValue`	used when the parameter is absent
`@Context`	`UriInfo`, `HttpHeaders`, `Request`, `SecurityContext`

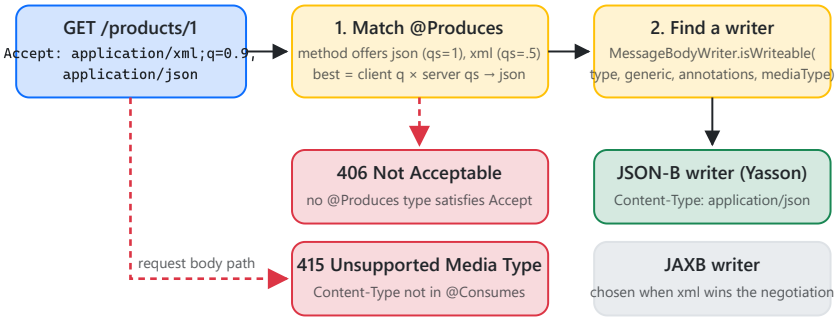
```
// Sub-resource locator: @Path, no HTTP
// method, returns the object that handles
// the rest of the path
@Path("{id}/reviews")
public ReviewResource reviews(
        @PathParam("id") long id) {
    return new ReviewResource(repo.require(id));
}

// handles /products/42/reviews and
// /products/42/reviews/7
public class ReviewResource {
    private final Product product;
    ReviewResource(Product p) { product = p; }

    @GET
    public List<ReviewDto> list() { ... }

    @GET @Path("{rid}")
    public ReviewDto one(
            @PathParam("rid") long rid) { ... }
}
```

Content negotiation: Accept → @Produces → MessageBodyWriter



Input is symmetric: Content-Type → @Consumes → MessageBodyReader runs before your method.

```java
@Path("/products")
public class ProductResource {

    // The writer is selected from Accept; the method body never changes
    @GET @Path("{id}")
    @Produces({MediaType.APPLICATION_JSON, MediaType.APPLICATION_XML})
    public ProductDto one(@PathParam("id") long id) {
        return service.get(id);
    }

    // Accept both bodies, always answer in JSON
    @POST
    @Consumes({MediaType.APPLICATION_JSON, MediaType.APPLICATION_XML})
    @Produces(MediaType.APPLICATION_JSON)
    public Response create(ProductDto dto, @Context UriInfo uriInfo) { ... }
}

// JSON-B 3 (Yasson) serialises records and classes without configuration
public record ReviewDto(
        long id,
        @JsonbProperty("author") String authorName,
        int stars,
        @JsonbDateFormat("yyyy-MM-dd") LocalDate written) {}

// XML needs a JAXB class (jersey-media-jaxb + jaxb-runtime); the same
// class also works for JSON-B, so one DTO can serve both formats
@XmlRootElement(name = "product")
@XmlAccessorType(XmlAccessType.FIELD)
public class ProductDto {
    @XmlAttribute long id;
    String name;
    String sku;
    BigDecimal price;
    public ProductDto() { }                // JAXB needs a no-arg constructor
    // getters, setters, static from(Product), toEntity()
}
```

Shaping representations: JSON-B configuration, server-side quality, vendor types

```java
// Customise JSON-B once for the whole application (a ContextResolver)
@Provider
public class JsonbConfigurer implements ContextResolver<Jsonb> {
    private final Jsonb jsonb = JsonbBuilder.create(new JsonbConfig()
            .withNullValues(false)                  // omit nulls
            .withFormatting(false)                  // compact output
            .withPropertyNamingStrategy(
                    PropertyNamingStrategy.LOWER_CASE_WITH_UNDERSCORES)
            .withDateFormat("yyyy-MM-dd'T'HH:mm:ssXXX", Locale.ROOT));

    @Override
    public Jsonb getContext(Class<?> type) { return jsonb; }
}

// Server-side preference: with Accept: */* the client gets JSON
@Produces({"application/json; qs=1.0", "application/xml; qs=0.5"})
public ProductDto one(@PathParam("id") long id) { ... }

// Vendor media types name the payload shape (and its version)
@GET @Path("{id}")
@Produces("application/vnd.shop.v2+json")
public ProductV2 oneV2(@PathParam("id") long id) { ... }

// Send the right error when nothing fits: the runtime does it for you
//   Accept: text/csv          → 406 Not Acceptable
//   Content-Type: text/plain  → 415 Unsupported Media Type
// Add "Vary: Accept" so caches keep JSON and XML copies apart.
```

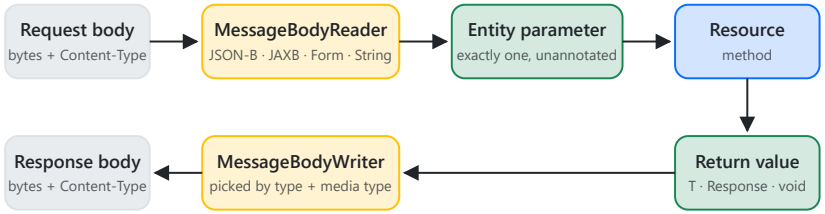
	JSON-B (Yasson)	JAXB	Jackson (Spring)
Standard	Jakarta JSON Binding 3.0	Jakarta XML Binding 4.0	de-facto library
Records	yes	no (needs class + no-arg ctor)	yes
Annotations	`@JsonbProperty`, `@JsonbTransient`	`@XmlRootElement`, `@XmlElement`	`@JsonProperty`, `@JsonIgnore`
Jersey module	`jersey-media-json-binding`	`jersey-media-jaxb`	`jersey-media-json-jackson`

```
$ curl -s localhost:8080/api/products/1 \
       -H 'Accept: application/xml'
<?xml version="1.0" encoding="UTF-8"?>
<product id="1">
  <name>Kettle</name><sku>KT-01</sku>
  <price>24.90</price>
</product>

$ curl -si localhost:8080/api/products/1 \
       -H 'Accept: text/csv'
HTTP/1.1 406 Not Acceptable
```

💡 JSON is the default for new APIs; keep XML only when a client needs it. Never let the entity classes from Chapter 03 leak out: map to DTOs.

Entities in, entities out: parameters, return types and the Response builder



Return kind → status

- `void` → 204 No Content
- `T` (non-null) → 200, body written
- `T` = null → 204
- `Response` → exactly what you built
- `CompletionStage<T>` → async, 200 when done
- `StreamingOutput` → 200, body streamed

```java
// The entity parameter is the one without annotations; @Context
// injects request-scoped helpers such as UriInfo
@POST
@Consumes(MediaType.APPLICATION_JSON)
public Response create(@Valid CreateProduct cmd, @Context UriInfo uriInfo) {
    Product saved = service.create(cmd);
    // /api/products  →  /api/products/{id}  →  /api/products/42
    URI self = uriInfo.getAbsolutePathBuilder().path("{id}").build(saved.id());
    return Response.created(self)              // 201 + Location: self
                   .entity(ProductDto.from(saved))
                   .build();
}

// Returning Response with a generic collection: keep the type for the
// writer (JAXB needs it; JSON-B copes, but be explicit)
@GET
public Response list() {
    List<ProductDto> items = service.findAll();
    return Response.ok(new GenericEntity<List<ProductDto>>(items) {})
                   .build();
}

// Large payloads: stream instead of building the whole body in memory
@GET @Path("export")
@Produces("text/csv")
public StreamingOutput export() {
    return out -> {
        try (var w = new PrintWriter(out, false, StandardCharsets.UTF_8)) {
            w.println("id,name,price");
            service.findAll().forEach(p ->
                    w.printf("%d,%s,%s%n", p.id(), p.name(), p.price()));
        }
    };
}
```

The same entities from the other side: `Client`, `WebTarget`, `Entity`

```
// Client is heavyweight: create one, reuse it, close it at shutdown
Client client = ClientBuilder.newClient();
WebTarget products = client.target("http://localhost:8080/api")
                           .path("products");          // targets are immutable

// GET a list: GenericType keeps List<ProductDto> through type erasure
List<ProductDto> books = products.queryParam("category", "books")
        .request(MediaType.APPLICATION_JSON)
        .get(new GenericType<List<ProductDto>>() {});

// POST an entity, inspect status, headers and body (Response is Closeable)
var cmd = new CreateProduct("Kettle", new BigDecimal("24.90"), "KT-01", 5);
try (Response r = products.request().post(Entity.json(cmd))) {
    System.out.println(r.getStatus());              // 201
    System.out.println(r.getLocation());            // .../api/products/7
    ProductDto created = r.readEntity(ProductDto.class);
}

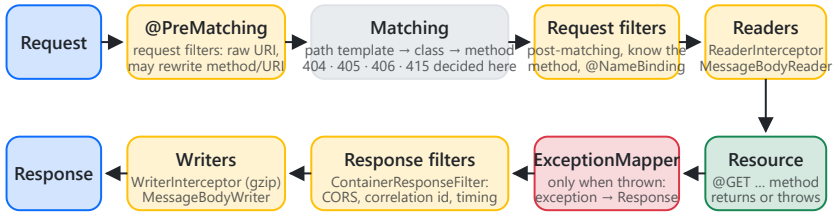
// Path templates, headers, conditional GET
try (Response r = products.path("{id}").resolveTemplate("id", 7)
        .request()
        .header(HttpHeaders.IF_NONE_MATCH, "\"v3\"")
        .get()) {
    if (r.getStatus() == 304) System.out.println("cached copy is current");
}

// Errors: get(Class) throws WebApplicationException for 4xx/5xx;
// get() returning Response never throws, you read the status yourself
client.close();
```

API	Purpose
`ClientBuilder.newClient()`	configure once: timeouts, providers, TLS
`WebTarget`	URI builder: `path`, `queryParam`, `resolveTemplate`
`request(mediaTypes)`	sets `Accept`, returns an `Invocation.Builder`
`Entity.json(x)`, `Entity.xml(x)`, `Entity.form(f)`, `Entity.entity(x, type)`	request body plus its `Content-Type`
`get(Class)`, `post(entity, Class)`	typed body, exceptions on errors
`readEntity(GenericType)`	typed generic body from a `Response`
`rx()`, `async()`	`CompletionStage` / `Future` based calls

⚠ Always close (or fully read) `Response` objects, otherwise the connection stays reserved. Register the same JSON-B and problem+json providers on the client that you use on the server.

Resources, providers and filters: the runtime pipeline



Rules of the pipeline

- Filters run in `@Priority` order (AUTHENTICATION 1000 < AUTHORIZATION 2000 < HEADER_DECORATOR 3000 < USER 5000).
- `ctx.abortWith(response)` in a request filter skips matching and the resource; response filters still run.
- Global providers apply everywhere; `@NameBinding` or a `DynamicFeature` restricts them to chosen methods.
- Providers are singletons: keep them stateless or thread-safe (Chapter 01).

```java
// Runs BEFORE matching: sees the raw request, can reject or rewrite it
@Provider @PreMatching
@Priority(Priorities.AUTHENTICATION)
public class ApiKeyFilter implements ContainerRequestFilter {
    @Override
    public void filter(ContainerRequestContext ctx) {
        if ("GET".equals(ctx.getMethod())) return;    // reads are public
        String key = ctx.getHeaderString("X-Api-Key");
        if (key == null || !ApiKeys.isValid(key)) {
            ctx.abortWith(Response.status(Response.Status.UNAUTHORIZED)
                    .header("WWW-Authenticate", "ApiKey realm=\"shop\"")
                    .build());
        }
    }
}

// Runs AFTER matching and again on the way out: one class, two roles
@Provider
@Priority(Priorities.USER)
public class CorrelationFilter implements ContainerRequestFilter,
                                          ContainerResponseFilter {
    private static final System.Logger log = System.getLogger("http");

    @Override
    public void filter(ContainerRequestContext req) {
        String id = req.getHeaderString("X-Correlation-Id");
        if (id == null) id = UUID.randomUUID().toString();
        req.setProperty("correlationId", id);      // request-scoped bag
    }

    @Override
    public void filter(ContainerRequestContext req,
                       ContainerResponseContext res) {
        String id = (String) req.getProperty("correlationId");
        res.getHeaders().putSingle("X-Correlation-Id", id);
        log.log(System.Logger.Level.INFO, "{0} {1} -> {2} [{3}]",
                req.getMethod(), req.getUriInfo().getPath(),
                res.getStatus(), id);
    }
}
```

CORS with a response filter, gzip with a `WriterInterceptor`, scoping with `@NameBinding`

```java
// CORS: browsers need these headers before they accept a cross-origin answer
@Provider
public class CorsFilter implements ContainerResponseFilter {
    @Override
    public void filter(ContainerRequestContext req,
                       ContainerResponseContext res) {
        var h = res.getHeaders();
        h.putSingle("Access-Control-Allow-Origin", "https://shop.example.com");
        h.putSingle("Access-Control-Allow-Methods", "GET,POST,PUT,DELETE");
        h.putSingle("Access-Control-Allow-Headers", "Content-Type,X-Api-Key");
        h.putSingle("Access-Control-Max-Age", "3600");
    }
}

// Name binding: a marker annotation that ties a provider to methods
@NameBinding
@Retention(RetentionPolicy.RUNTIME)
@Target({ElementType.TYPE, ElementType.METHOD})
public @interface Compress {}

@Provider @Compress
public class GzipWriterInterceptor implements WriterInterceptor {
    @Override
    public void aroundWriteTo(WriterInterceptorContext ctx) throws IOException {
        ctx.getHeaders().putSingle(HttpHeaders.CONTENT_ENCODING, "gzip");
        ctx.setOutputStream(new GZIPOutputStream(ctx.getOutputStream()));
        ctx.proceed();                    // the writer now writes into gzip
    }
}

// Only this method is compressed (check Accept-Encoding in the lab)
@GET @Compress
public List<ProductDto> list() { ... }
```

Provider type	When it runs / what it does
`ContainerRequestFilter`	before the resource: auth, logging, rewriting, `abortWith`
`ContainerResponseFilter`	after the resource: headers (CORS, cache), status changes
`ReaderInterceptor` / `WriterInterceptor`	around the body: decompress, compress, sign, log payload
`MessageBodyReader` / `Writer`	bytes ↔ objects per media type (JSON-B, JAXB, CSV…)
`ExceptionMapper<E>`	exception → `Response` (Topic 7)
`ContextResolver<T>`	supplies configured helpers (`Jsonb`, `JAXBContext`)
`ParamConverterProvider`	String ↔ parameter type (Topic 3)
`DynamicFeature` / `Feature`	register providers per method / bundle a set of providers

```java
// Preflight: the browser sends OPTIONS first.
// Answer it without hitting a resource:
@Provider @PreMatching
public class PreflightFilter
        implements ContainerRequestFilter {
    public void filter(ContainerRequestContext c) {
        if ("OPTIONS".equals(c.getMethod()))
            c.abortWith(Response.ok().build());
    }
}
```

⚠ CORS is a browser convenience, not a security boundary: curl ignores it. `Allow-Origin: *` together with credentials is refused by browsers.

The same endpoint in Jakarta REST and in Spring MVC (Chapter 02)

```java
// Jakarta REST 4.0 (Jersey)
@Path("/products")
@Produces(MediaType.APPLICATION_JSON)
public class ProductResource {
    @Inject ProductService service;

    @GET @Path("{id}")
    public ProductDto one(@PathParam("id") long id) {
        return service.get(id);
    }

    @POST
    @Consumes(MediaType.APPLICATION_JSON)
    public Response create(@Valid CreateProduct cmd,
                           @Context UriInfo info) {
        var saved = service.create(cmd);
        var uri = info.getAbsolutePathBuilder()
                      .path(Long.toString(saved.id())).build();
        return Response.created(uri).entity(saved).build();
    }
}
```

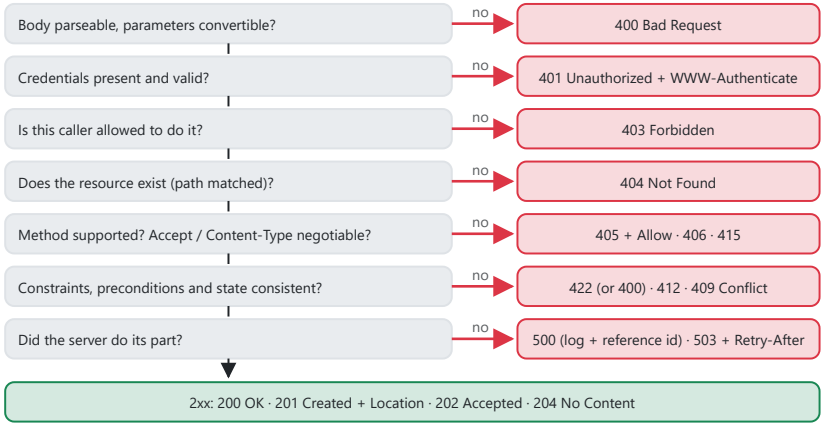
```java
// Spring MVC 7 (Spring Boot 4, spring-boot-starter-webmvc)
@RestController
@RequestMapping(path = "/products",
                produces = MediaType.APPLICATION_JSON_VALUE)
public class ProductController {
    private final ProductService service;
    ProductController(ProductService s) { this.service = s; }

    @GetMapping("/{id}")
    public ProductDto one(@PathVariable long id) {
        return service.get(id);
    }

    @PostMapping(consumes = MediaType.APPLICATION_JSON_VALUE)
    public ResponseEntity<ProductDto> create(
            @RequestBody @Valid CreateProduct cmd) {
        var saved = service.create(cmd);
        var uri = ServletUriComponentsBuilder.fromCurrentRequest()
                .path("/{id}").build(saved.id());
        return ResponseEntity.created(uri).body(saved);
    }
}
```

	Jakarta REST 4.0 / Jersey	**Spring MVC 7**
Origin	Jakarta EE specification, several implementations (Jersey, RESTEasy, Open Liberty)	Spring project, one implementation
Path / method	`@Path` + `@GET`, `@POST`…	`@RequestMapping`, `@GetMapping`, `@PostMapping`…
Parameters	`@PathParam`, `@QueryParam`, `@HeaderParam`, `@BeanParam`; body = the unannotated parameter	`@PathVariable`, `@RequestParam`, `@RequestHeader`, `@ModelAttribute`; body = `@RequestBody`
Response	`Response` builder, `UriInfo`	`ResponseEntity`, `@ResponseStatus`, `ServletUriComponentsBuilder`
Errors	`ExceptionMapper` providers; `WebApplicationException`	`@RestControllerAdvice` + `@ExceptionHandler`; `ProblemDetail` built in
Cross-cutting	`ContainerRequest/ResponseFilter`, interceptors, `@NameBinding`	Servlet filters, `HandlerInterceptor`, AOP
JSON / DI / tests	JSON-B by default (Jackson optional) / HK2 or CDI / `JerseyTest`	Jackson / Spring container / `@WebMvcTest` + MockMvc

One error shape for everything: `ExceptionMapper` and RFC 9457 problem details



The runtime decides 400 (conversion), 404, 405, 406 and 415 for you; the rest is your code and your mappers.

```java
// RFC 9457 "problem details": type, title, status, detail, instance
public record Problem(String type, String title, int status,
                      String detail, String instance) {
    public static final String MEDIA_TYPE = "application/problem+json";
}

// One mapper per exception family; the closest type wins
@Provider
public class NotFoundMapper implements ExceptionMapper<NotFoundException> {
    @Context UriInfo uriInfo;                // providers can use @Context

    @Override
    public Response toResponse(NotFoundException ex) {
        var problem = new Problem("about:blank", "Not Found", 404,
                ex.getMessage(), uriInfo.getRequestUri().getPath());
        return Response.status(404).type(Problem.MEDIA_TYPE)
                       .entity(problem).build();
    }
}

// Catch-all: never leak a stack trace; log it with a reference id
@Provider
public class UnexpectedMapper implements ExceptionMapper<Throwable> {
    private static final System.Logger log = System.getLogger("api");

    @Override
    public Response toResponse(Throwable ex) {
        if (ex instanceof WebApplicationException w)
            return w.getResponse();       // keep 4xx/5xx thrown on purpose
        String ref = UUID.randomUUID().toString();
        log.log(System.Logger.Level.ERROR, "ref=" + ref, ex);
        var problem = new Problem("about:blank", "Internal Server Error",
                500, "Unexpected error, reference " + ref, null);
        return Response.serverError().type(Problem.MEDIA_TYPE)
                       .entity(problem).build();
    }
}
```

Validate at the boundary: `@Valid`, constraints on parameters, one mapper for violations

```java
// Constraints on the DTO (jersey-bean-validation + Hibernate Validator 9)
public record CreateProduct(
        @NotBlank @Size(max = 80) String name,
        @NotNull @DecimalMin("0.01") @Digits(integer = 8, fraction = 2)
        BigDecimal price,
        @NotNull @Pattern(regexp = "[A-Z]{2}-\\d{2}") String sku,
        @PositiveOrZero int stock) {}

@Path("/products")
public class ProductResource {

    // @Valid cascades into the entity; a missing body is @NotNull's job
    @POST
    @Consumes(MediaType.APPLICATION_JSON)
    public Response create(@Valid @NotNull CreateProduct cmd) { ... }

    // Constraints directly on parameters
    @GET
    public List<ProductDto> page(
            @QueryParam("page") @DefaultValue("0") @Min(0) int page,
            @QueryParam("size") @DefaultValue("20")
            @Min(1) @Max(100) int size) { ... }

    // Return values can be validated too (catches server-side bugs)
    @GET @Path("{id}")
    public @Valid ProductDto one(@PathParam("id") long id) { ... }
}
```

```java
// Replace Jersey's default 400 text with problem+json and a field list
@Provider
public class ValidationMapper
        implements ExceptionMapper<ConstraintViolationException> {

    @Override
    public Response toResponse(ConstraintViolationException ex) {
        var errors = ex.getConstraintViolations().stream()
                .map(v -> Map.of("field", lastNode(v.getPropertyPath()),
                                 "message", v.getMessage()))
                .toList();
        var body = Map.of("type", "about:blank",
                          "title", "Invalid request", "status", 400,
                          "detail", "Validation failed", "errors", errors);
        return Response.status(400).type(Problem.MEDIA_TYPE)
                       .entity(body).build();
    }

    // "create.cmd.price" → "price"
    private static String lastNode(Path path) {
        String name = null;
        for (Path.Node n : path) name = n.getName();
        return name;
    }
}
```

```
HTTP/1.1 400 Bad Request
Content-Type: application/problem+json

{ "type": "about:blank", "title": "Invalid request", "status": 400,
  "detail": "Validation failed",
  "errors": [
    { "field": "price", "message": "must be greater than or equal to 0.01" },
    { "field": "sku",   "message": "must match \"[A-Z]{2}-\\d{2}\"" } ] }
```

Pagination: `page`/`size` parameters, `Link` headers, total count

GET /api/products?page=2&size=20 (187 items → pages 0…9)




```
HTTP/1.1 200 OK
X-Total-Count: 187                                          RFC 8288 web linking
Link: </api/products?size=20&page=0>; rel="first",
</api/products?size=20&page=1>; rel="prev",
</api/products?size=20&page=3>; rel="next",
</api/products?size=20&page=9>; rel="last"
```

```java
@GET
public Response page(@QueryParam("page") @DefaultValue("0") @Min(0) int page,
                     @QueryParam("size") @DefaultValue("20")
                     @Min(1) @Max(100) int size,
                     @Context UriInfo uriInfo) {
    Page<ProductDto> result = service.page(page, size);

    // Build links from the request URI: never hard-code host or path
    UriBuilder base = uriInfo.getAbsolutePathBuilder()
                             .queryParam("size", size);
    var links = new ArrayList<Link>();
    links.add(link(base, 0, "first"));
    if (page > 0)                links.add(link(base, page - 1, "prev"));
    if (page < result.lastPage()) links.add(link(base, page + 1, "next"));
    links.add(link(base, result.lastPage(), "last"));

    return Response.ok(result.items())
                   .links(links.toArray(Link[]::new))     // Link: header(s)
                   .header("X-Total-Count", result.total())
                   .build();
}

private static Link link(UriBuilder base, int page, String rel) {
    return Link.fromUriBuilder(base.clone().queryParam("page", page))
               .rel(rel).build();
}

// Alternative for large or fast-changing data: cursor (keyset) paging
//   GET /products?after=eyJpZCI6NDJ9&size=20  → Link rel="next" only
// Offsets skip rows on every page; a cursor resumes after the last key.
```

Filtering and sorting safely, versioning deliberately

```java
// Filtering and sorting: whitelist fields, never forward them to a query
private static final Set<String> SORTABLE = Set.of("name", "price", "since");

@GET
public List<ProductDto> search(@BeanParam @Valid ProductFilter f) {
    // ?sort=price,desc
    String[] sort = f.sort().split(",");
    if (!SORTABLE.contains(sort[0]))
        throw new BadRequestException("Unknown sort field " + sort[0]);
    return service.search(f, sort[0], sort.length > 1 && "desc".equals(sort[1]));
}

// Versioning option A: in the URI (visible, easy with curl and caches)
@Path("/v1/products") public class ProductResourceV1 { ... }
@Path("/v2/products") public class ProductResourceV2 { ... }

// Versioning option B: in the media type (one URI per resource, negotiated)
@Path("/products")
public class ProductResource {

    @GET @Path("{id}")
    @Produces({"application/vnd.shop.v1+json", MediaType.APPLICATION_JSON})
    public ProductV1 v1(@PathParam("id") long id) {
        return ProductV1.from(service.get(id));
    }

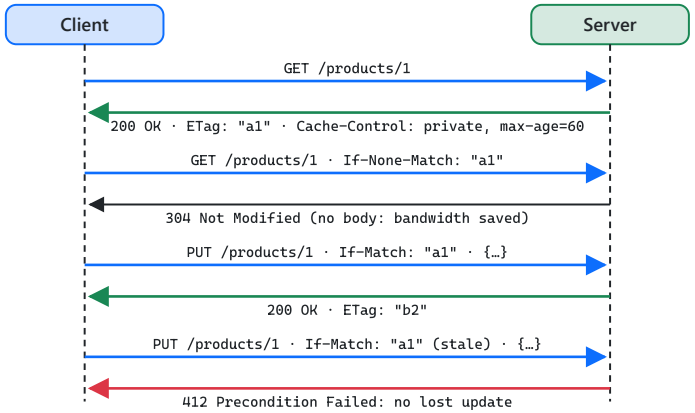
    // v2 changed price from a number to {"amount": "24.90", "currency": "EUR"}
    @GET @Path("{id}")
    @Produces("application/vnd.shop.v2+json")
    public ProductV2 v2(@PathParam("id") long id) {
        return ProductV2.from(service.get(id));
    }
}
```

Where	Example	Pros / cons
URI path	`/v2/products/1`	obvious, cache-friendly, browsable; the "same" resource has two URIs
Query param	`/products/1?v=2`	easy to add; easy to forget, pollutes caches
Custom header	`Api-Version: 2`	clean URIs; invisible in links, needs `Vary`
Media type	`Accept: application/vnd.shop.v2+json`	pure HTTP negotiation, per-representation versions; hardest to explain to consumers

```
$ curl -s localhost:8080/api/products/1 \
    -H 'Accept: application/vnd.shop.v2+json'
{"id":1,"name":"Kettle",
 "price":{"amount":"24.90","currency":"EUR"}}
```

Most changes need no version at all: adding optional fields is backwards compatible. Version when you rename, remove or change the meaning of a field, and support the old version for a stated period.

Caching and conditional requests: ETag, If-None-Match, If-Match, Cache-Control



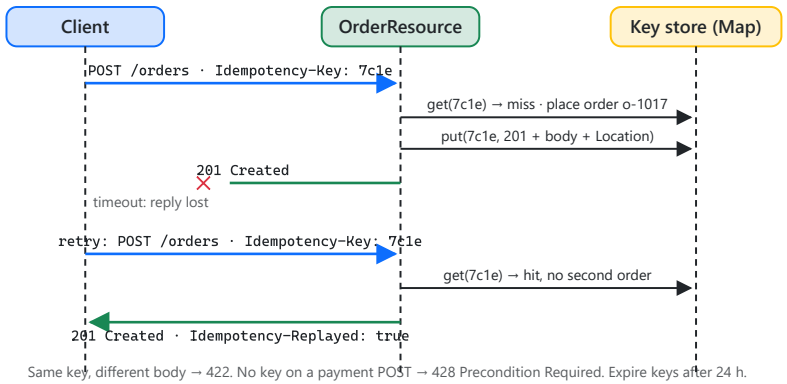

```java
@GET @Path("{id}")
public Response one(@PathParam("id") long id, @Context Request request) {
    Product p = service.get(id);
    EntityTag tag = new EntityTag(Integer.toHexString(p.version()));

    // If-None-Match matches → 304 builder; If-Match fails → 412 builder
    Response.ResponseBuilder cond = request.evaluatePreconditions(tag);
    if (cond != null) return cond.tag(tag).build();

    CacheControl cc = new CacheControl();
    cc.setMaxAge(60);            // seconds a cache may reuse the copy
    cc.setPrivate(true);         // browser cache only, no shared proxy
    cc.setNoTransform(true);
    return Response.ok(ProductDto.from(p))
                   .tag(tag)
                   .cacheControl(cc)
                   .lastModified(Date.from(p.updatedAt()))
                   .build();
}

// Optimistic locking over HTTP: require If-Match on updates
@PUT @Path("{id}")
public Response replace(@PathParam("id") long id, @Valid ProductDto dto,
                        @HeaderParam(HttpHeaders.IF_MATCH) String ifMatch,
                        @Context Request request) {
    if (ifMatch == null)
        throw new ClientErrorException("If-Match required", 428);
    Product current = service.get(id);
    EntityTag tag = new EntityTag(Integer.toHexString(current.version()));
    Response.ResponseBuilder cond = request.evaluatePreconditions(tag);
    if (cond != null) return cond.build();               // 412
    Product saved = service.replace(id, dto);
    return Response.ok(ProductDto.from(saved))
                   .tag(new EntityTag(Integer.toHexString(saved.version())))
                   .build();
}
```

Idempotency: safe retries for POST with an `Idempotency-Key`



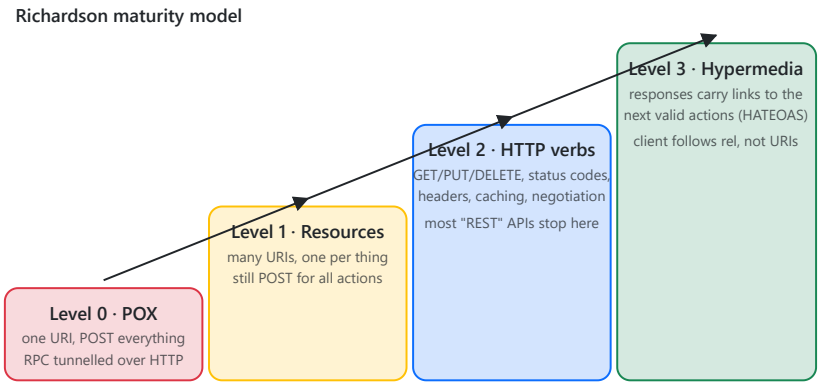

```java
// GET, PUT and DELETE are idempotent by definition: repeating them
// changes nothing more. POST is not: a retry after a timeout would
// create a second order. The client sends a unique key per attempt.
@Path("/orders")
public class OrderResource {
    private record Stored(int status, URI location, Object body) {}
    private final ConcurrentMap<String, Stored> seen;   // injected singleton

    @POST
    @Consumes(MediaType.APPLICATION_JSON)
    public Response place(@HeaderParam("Idempotency-Key") String key,
                          @Valid PlaceOrder cmd, @Context UriInfo uriInfo) {
        if (key == null || key.isBlank())
            throw new ClientErrorException("Idempotency-Key required", 428);

        boolean[] replayed = { true };
        // computeIfAbsent runs the mapping function at most once per key,
        // even when two retries arrive at the same moment (Chapter 01)
        Stored s = seen.computeIfAbsent(key, k -> {
            replayed[0] = false;
            Order order = service.place(cmd);
            URI loc = uriInfo.getAbsolutePathBuilder().path(order.id()).build();
            return new Stored(201, loc, OrderDto.from(order));
        });
        return Response.status(s.status())
                       .location(s.location())
                       .header("Idempotency-Replayed", replayed[0])
                       .entity(s.body())
                       .build();
    }
}

// Production: store the key with a hash of the body and a TTL (Redis,
// database table), and reject a reused key with a different body (422).
```

HATEOAS: the response tells the client what it can do next

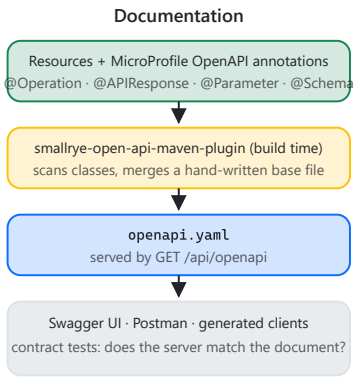


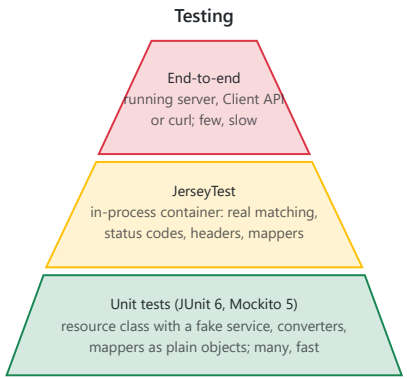

```
@GET @Path("{id}")
@Produces(MediaType.APPLICATION_JSON)
public Response one(@PathParam("id") String id, @Context UriInfo uriInfo) {
    Order order = service.get(id);
    UriBuilder self = uriInfo.getAbsolutePathBuilder();

    // Link headers (RFC 8288): only the transitions valid in this state
    var links = new ArrayList<Link>();
    links.add(Link.fromUriBuilder(self).rel("self").build());
    if (order.status() == OrderStatus.NEW) {
        links.add(Link.fromUriBuilder(self.clone().path("payment"))
                      .rel("pay").type("application/json").build());
        links.add(Link.fromUriBuilder(self.clone())
                      .rel("cancel").param("method", "DELETE").build());
    }
    // The same links inside the body, for clients that ignore headers
    Map<String, String> bodyLinks = links.stream().collect(
            Collectors.toMap(Link::getRel, l -> l.getUri().toString()));
    return Response.ok(OrderDto.from(order, bodyLinks))
                   .links(links.toArray(Link[]::new))
                   .build();
}
```

```
{ "id": "o-1017", "status": "NEW", "total": "49.80",
  "_links": { "self":   "http://localhost:8080/api/orders/o-1017",
              "pay":    "http://localhost:8080/api/orders/o-1017/payment",
              "cancel": "http://localhost:8080/api/orders/o-1017" } }
```

Documenting the API: OpenAPI 3.1 from annotations






```java
// org.eclipse.microprofile.openapi:microprofile-openapi-api (annotations)
// io.smallrye:smallrye-open-api-maven-plugin writes target/openapi.yaml
@Path("/products")
@Tag(name = "Products", description = "Catalogue of sellable items")
public class ProductResource {

    @GET @Path("{id}")
    @Produces(MediaType.APPLICATION_JSON)
    @Operation(summary = "Get one product",
               description = "Returns the product with the given id")
    @APIResponse(responseCode = "200", description = "The product",
                 content = @Content(schema = @Schema(
                         implementation = ProductDto.class)))
    @APIResponse(responseCode = "404", description = "Unknown id",
                 content = @Content(mediaType = "application/problem+json",
                         schema = @Schema(implementation = Problem.class)))
    public ProductDto one(
            @Parameter(description = "Product id", example = "42")
            @PathParam("id") long id) {
        return service.get(id);
    }
}
```

```yaml
# excerpt of the generated document
openapi: 3.1.0
paths:
  /products/{id}:
    get:
      tags: [Products]
      summary: Get one product
      parameters:
        - { name: id, in: path, required: true, schema: { type: integer } }
      responses:
        "200": { description: The product }
        "404": { description: Unknown id }
```

Testing resources: `JerseyTest` with JUnit 6

```
// test scope: org.glassfish.jersey.test-framework.providers:
//   jersey-test-framework-provider-grizzly2 (starts a real HTTP server
//   on a free port for each test class) + org.junit:junit-bom 6.1.3
class ProductResourceTest extends JerseyTest {

    // The application under test: only the providers this test needs
    @Override
    protected Application configure() {
        return new ResourceConfig(ProductResource.class,
                                  NotFoundMapper.class,
                                  ValidationMapper.class)
                .register(new TestBinder());     // fake repository, 1 product
    }

    @Test
    void getExistingProductReturns200() {
        Response r = target("products/1").request().get();
        assertEquals(200, r.getStatus());
        assertEquals("Kettle", r.readEntity(ProductDto.class).name());
    }

    @Test
    void unknownIdReturns404ProblemJson() {
        Response r = target("products/999").request().get();
        assertEquals(404, r.getStatus());
        assertEquals("application/problem+json",
                     r.getMediaType().toString());
    }

    @Test
    void createReturns201WithLocation() {
        var cmd = new CreateProduct("Mug", new BigDecimal("6.50"), "MG-01", 3);
        Response r = target("products").request().post(Entity.json(cmd));
        assertEquals(201, r.getStatus());
        assertTrue(r.getLocation().getPath().endsWith("/products/2"));
    }

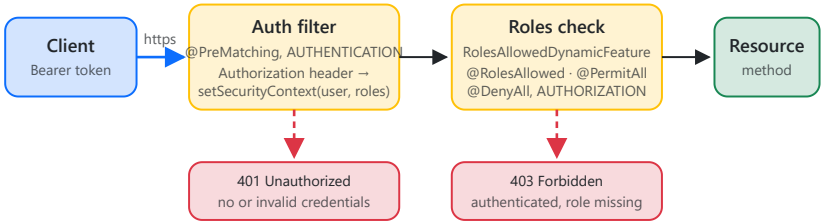
    @Test
    void invalidBodyReturns400WithFieldErrors() {
        var cmd = new CreateProduct("", new BigDecimal("0"), "bad", -1);
        Response r = target("products").request().post(Entity.json(cmd));
        assertEquals(400, r.getStatus());
        assertEquals(4, r.readEntity(JsonObject.class)
                         .getJsonArray("errors").size());
    }
}
```

Level	What you test	How
Unit	mapping, converters, business rules	plain JUnit 6; `new ProductResource(fakeRepo).one(1)`; Mockito 5 for collaborators
Container	matching, status codes, headers, negotiation, mappers, filters	`JerseyTest`: `configure()` returns the app, `target(...)` gives a client
End-to-end	the deployed artifact, TLS, real data	`ClientBuilder.newClient()` against a started server, or curl in CI
Contract	server matches `openapi.yaml`	schema validation of responses in the container tests

- `JerseyTest` wires `setUp()`/`tearDown()` with Jupiter's `@BeforeEach`/`@AfterEach`; JUnit 6 keeps the Jupiter API, so import the `junit-bom` and let it override the framework's transitive version.
- Set `-Djersey.config.test.container.port=0` for a random port, so tests run in parallel with a dev server.
- Register the same JSON provider in the test client: `configureClient(ClientConfig c)`.
- One test per status code you promise in the OpenAPI document: 200, 201, 204, 304, 400, 401, 404, 406, 409, 412, 415.

Chapter 06 goes deeper into JUnit 6 and Mockito; here the goal is that every promise the API makes is executable.

Security basics: TLS, the `Authorization` header, `SecurityContext`, `@RolesAllowed`



- TLS everywhere: tokens and API keys travel in headers, never in the query string (logs, referrers, history).
- Validate every input, limit body size and page size, rate-limit (429), keep error details generic.
- Chapter 05 replaces the token map with Spring Security and signed JWTs; the HTTP rules stay the same.

```java
// 1. Authenticate: turn the Authorization header into a SecurityContext
@Provider @PreMatching @Priority(Priorities.AUTHENTICATION)
public class BearerAuthFilter implements ContainerRequestFilter {
    @Override
    public void filter(ContainerRequestContext ctx) {
        String auth = ctx.getHeaderString(HttpHeaders.AUTHORIZATION);
        User user = auth != null && auth.startsWith("Bearer ")
                ? Tokens.lookup(auth.substring(7)) : null;
        if (user == null) return;            // anonymous: the roles decide
        boolean secure = ctx.getSecurityContext().isSecure();
        ctx.setSecurityContext(new SecurityContext() {
            public Principal getUserPrincipal() { return user::name; }
            public boolean isUserInRole(String role) {
                return user.roles().contains(role);
            }
            public boolean isSecure() { return secure; }
            public String getAuthenticationScheme() { return "Bearer"; }
        });
    }
}

// 2. Authorise with jakarta.annotation.security on the resource
@Path("/products")
public class ProductResource {
    @GET @PermitAll
    public List<ProductDto> list() { ... }

    // 401 when anonymous, 403 when authenticated without the role
    @POST @RolesAllowed("admin")
    public Response create(@Valid CreateProduct cmd) { ... }

    @DELETE @Path("{id}") @RolesAllowed({"admin", "ops"})
    public void delete(@PathParam("id") long id,
                       @Context SecurityContext sc) {
        audit.log(sc.getUserPrincipal().getName(), "delete", id);
    }
}

// 3. Enable the checks (Jersey): config.register(RolesAllowedDynamicFeature.class)
```

WRAP-UP

Best practices and common mistakes

1 Nouns in URIs, verbs in methods

Plural, lower-case, hyphenated collections; items by id; sub-collections for containment; state transitions as resources (`/orders/1/payment`), not `/pay`.

2 Let the status code tell the truth

201 + `Location` for creation, 204 for no body, 404 vs 400 vs 409 vs 422 by cause. Build URIs with `UriInfo`, never with string concatenation.

3 One error shape

`ExceptionMapper`s produce `application/problem+json` for every failure, including validation. Server errors are logged with a reference id and never leak stack traces.

4 Thin, stateless resources

Resources map HTTP to a service call and back. They are per-request objects; providers are singletons. State lives in injected, thread-safe components (Chapter 01).

5 Cache-friendly and retry-safe

`ETag` + `Cache-Control` on GETs, `If-Match` required on updates, `Idempotency-Key` on POSTs that cost money. Paginate everything that can grow.

6 Document, test, secure

OpenAPI generated from annotations; one `JerseyTest` per promised status code; TLS, `Authorization` header, `@RolesAllowed`, validated input, sane limits.

Common mistakes: `200 OK` with `{"error": …}` in the body · verbs in paths (`/getProducts`, `/products/1/delete`) · mutable fields in a per-request resource class, or a shared `HashMap` in a singleton provider · returning JPA entities directly (lazy-loading exceptions, leaked columns) · swallowing exceptions in the resource instead of mapping them · forgetting `Vary: Accept` when negotiating · accepting `sort` or filter fields unchecked · hard-coded `http://localhost:8080` in `Location` and `Link` headers.

Chapter quiz 10 questions

1. Which HTTP method is both safe and idempotent?

- POST
- GET
- PATCH
- None of them

2. A POST creates a new product. Which response is correct?

- 200 OK with the product
- 204 No Content
- 201 Created with a Location header pointing at the new resource
- 202 Accepted

3. Which pair binds ?size=20 to a parameter and supplies 20 when it is absent?

- @PathParam + @Context
- @QueryParam + @DefaultValue
- @HeaderParam + @BeanParam
- @FormParam + @Produces

4. The client sends Accept: text/csv but the method only has @Produces("application/json"). What happens?

- JSON is returned anyway
- 415 Unsupported Media Type
- 406 Not Acceptable
- 404 Not Found

5. Which provider turns an exception into an HTTP response?

- ContainerResponseFilter
- MessageBodyWriter
- ExceptionMapper
- ParamConverterProvider

6. A ContainerRequestFilter must inspect the raw URI before the resource method is chosen. Which annotation does it need?

- @PreMatching
- @NameBinding
- @Priority
- @Context

7. What does Request.evaluatePreconditions(tag) return when the request carries a matching If-None-Match header?

- null
- A ResponseBuilder already set to 304 Not Modified
- A ResponseBuilder set to 412
- It throws NotModifiedException

8. Which media type does RFC 9457 define for error bodies?

- application/error+json
- application/problem+json
- text/problem
- application/vnd.error

9. A method annotated with @Path("{id}/reviews") but with no HTTP method annotation that returns a ReviewResource is called a...

- sub-resource locator
- name-bound filter
- dynamic feature
- entity provider

10. Which request selects version 2 of a representation through content negotiation rather than the URI?

- GET /v2/products/1
- GET /products/1?v=2
- GET /products/1 with Accept: application/vnd.shop.v2+json
- GET /products/1 with X-Api-Key: v2

WRAP-UP

Summary

Resources are nouns addressed by URIs; the HTTP methods are the verbs. Jakarta REST 4.0 maps both to Java with `@Path` and `@GET`… `@DELETE`.

Status codes tell the truth: 201 + `Location`, 204, 4xx for the client's fault, 5xx for ours; headers carry negotiation, caching and auth.

`@PathParam`, `@QueryParam` + `@DefaultValue`, `@BeanParam`, `ParamConverter` and sub-resource locators bind URIs to Java.

`Accept` and `Content-Type` meet `@Produces`/`@Consumes`; a `MessageBodyWriter`/`Reader` (JSON-B, JAXB) does the conversion, else 406/415.

The `Response` builder and `UriInfo` shape entities and headers; the `Client` API consumes them with `WebTarget` and `Entity`.

Filters and interceptors form the pipeline; `ExceptionMapper`s give one problem+json error shape for every failure.

Bean Validation on DTOs and parameters, with a `ConstraintViolationException` mapper returning 400 and a field list.

Paginate with `Link` headers, whitelist filters, version through URI or vendor media type; cache with `ETag` and `Cache-Control`, update with `If-Match`.

Idempotency keys make POST retry-safe, HATEOAS links guide clients; document with OpenAPI, test with `JerseyTest`, secure with TLS and `@RolesAllowed`.

Open Lab-04: 5 tasks + 1 challenge

Next chapter: 05 · Security in Java Web Applications (Spring Security, JWT)